

Playing Audio Songs Using Binary File Handling

27 July 2026 20:24

A binary file stores data in the same format as it exist in memory. Unlike text files, binary files are not human readable.

For example:

- Mp3 files
- Wav files
- Images
- Videos
- Executable files
- PDF Files
- And many more.

An audio file consist of thousands (or millions) of bytes representing sound samples.

Example:

Song.wav

01101110

11100010

10011001

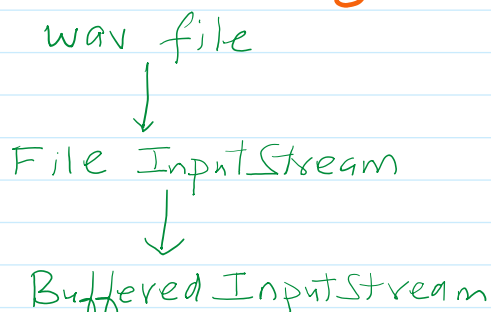
...

Java reads these bytes using streams.

Java Classes Used

Class	Purpose
File	Represents the audio file.
FileInputStream	Reads binary bytes.
BufferedInputStream	Faster binary reading.
ByteArrayOutputStream	Stores bytes in memory.
AudioInputStream	Convert bytes into audio stream.
AudioSystem	Plays sound.
Clip	Controls playback.

Binary File Reading Process.





```
import java.io.*;
import javax.sound.sampled.*;
/**
 * Write a description of class PlayAudio here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class PlayAudio
{
    public static void main(String[] args){
        String path = "./music/summermusic/summer-song-1.wav";
        File file = new File(path);
        try
        {
            AudioInputStream audio = AudioSystem.getAudioInputStream(file);
            Clip clip = AudioSystem.getClip();
            clip.open(audio);
            clip.start();
            System.out.print("\nPlaying...");
            Thread.sleep(clip.getMicrosecondLength()/1000);
        }
        catch (Exception uafe)
        {
            uafe.printStackTrace();
        }
    }
}
```

Another way: Play audio after reading Bytes

```
import java.io.*;
import javax.sound.sampled.*;
/**
 * Write a description of class PlayUsingBinay here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class PlayUsingBinary
{
    public static void main(String[] args) {
        String path = "./music/summermusic/summer-song-1.wav";
        File file = new File(path);
        try{
            FileInputStream fis = new FileInputStream(file);
            ByteArrayOutputStream baos = new ByteArrayOutputStream();

            byte buffer[] = new byte[4096];
            int bytesRead;
            while((bytesRead=fis.read(buffer))!=-1){
                baos.write(buffer, 0, bytesRead);
            }

            fis.close();
            byte audioData[] = baos.toByteArray();

            ByteArrayInputStream bis = new ByteArrayInputStream(audioData);
            AudioInputStream ais = AudioSystem.getAudioInputStream(bis);

            Clip clip = AudioSystem.getClip();
            clip.open(ais);
            clip.start();
            System.out.print("\nPlaying from binary data...");
            Thread.sleep(clip.getMicrosecondLength()/1000);
        }
        catch(FileNotFoundException fnfe){
            System.err.append("\nFile not found");
        }
        catch(IOException e){
            System.err.print("\nNetwork error!");
        }
        catch(UnsupportedAudioFileException ue){
            System.err.print("\nAudio format not supported exception");
        }
        catch(LineUnavailableException lue){
            System.out.print("\nLine is not available.");
        }
        catch(InterruptedException e){
            System.err.print("\nTimer is interrupted.");
        }
    }
}
```

```
}  
}
```

Added menu to play, stop and exit:

```
import java.io.*;  
import javax.sound.sampled.*;  
import java.util.*;  
  
/**  
 * Write a description of class PlayUsingBinay here.  
 *  
 * @author (your name)  
 * @version (a version number or a date)  
 */  
public class PlayUsingBinary  
{  
    public static void main(String[] args) {  
        String path = "./music/summermusic/summer-song-1.wav";  
        File file = new File(path);  
        try{  
            FileInputStream fis = new FileInputStream(file);  
            ByteArrayOutputStream baos = new ByteArrayOutputStream();  
  
            byte buffer[] = new byte[4096];  
            int bytesRead;  
            while((bytesRead=fis.read(buffer))!=-1){  
                baos.write(buffer, 0, bytesRead);  
            }  
  
            fis.close();  
            byte audioData[] = baos.toByteArray();  
  
            ByteArrayInputStream bis = new ByteArrayInputStream(audioData);  
            AudioInputStream ais = AudioSystem.getAudioInputStream(bis);  
  
            Clip clip = AudioSystem.getClip();  
            clip.open(ais);  
            System.out.print("\f");  
            Scanner sc = new Scanner(System.in);  
            while(true){  
                System.out.print("\n1.Play\n2.Stop\n3.exit\n"+  
                    "Enter your choice: ");  
                int ch = sc.nextInt();  
                switch(ch){  
                    case 1:  
                        clip.setFramePosition(0);  
                        clip.start();  
                        break;  
                    case 2:  
                        clip.stop();
```



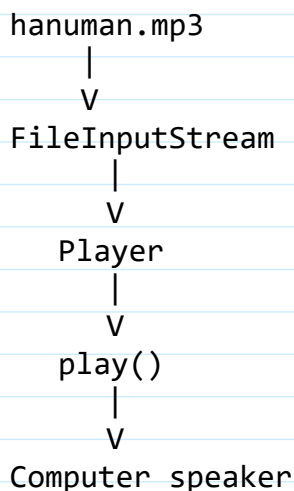
```
import javazoom.jl.decoder.*;
```

Step-5 Write the Program:

```
import java.io.*;
import javazoom.jl.player.Player;
import javazoom.jl.decoder.*;
/**
 * Write a description of class PlayerDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class PlayerDemo
{
    public static void main(String[] args){
        // "C:\Users\bluej\Documents\Students\Aadit Goenka\Java\Github\Coding
        \MP3Player\Songs\Collection1\hanuman.mp3"
        String fileName = "./Songs/Collection1/hanuman.mp3";
        File file = new File(fileName);
        System.out.print("\f");
        try(FileInputStream fis = new FileInputStream(file)){
            Player player = new Player(fis);
            System.out.print("\nPlaying...");
            player.play();
            System.out.print("\nPlayback finished");
        }
        catch(IOException e){

        }
        catch(JavaLayerException e){

        }
    }
}
```



How Player.play() works?

When you call:
 `player.play();`

Jlayer:

- a. Reads MP3 data from the file.
- b. Decodes the compressed MP3 frames.
- c. Convert them into PCM (Pulse code modulation) audio.
- d. Since the decoded audio to the system sound device.